

RTL Design Sherpa

RAPIDS Characterization Nexys A7 Project Guide 0.91

July 2026

Table of Contents

1 Document Information....8
1.1 References....9
1.2 Terminology....10
1.3 Revision History....11
2 Overview....12
2.1 What it characterizes....12
2.2 Boards and tools....13
2.3 Datapath geometry....14
2.4 Status....15
3 How It Works....16
3.1 The spine and region decode....16
3.1.1 Figure 2.1: RAPIDS Characterization Harness Spine....16
3.2 Key RTL blocks....18
3.3 The DUT and the on-chip harness....19
3.4 Atomic launch....20
4 Test Methodology....21
4.1 What is under test....22
4.2 What is measured, and how....23
4.2.1 Figure 3.1: Measurement Pipeline....24
4.3 Workloads....25
4.4 The oracle....26
4.5 Measurement pitfalls....27

5 Build and Run....28
5.1 Make targets (flows-rapids-beats/Makefile)....29
5.2 The host campaign (flows-rapids-beats/host/)....31
5.3 Typical sequence....32
6 Harness CSR Configuration....33
6.1 Regions and by-name access....33
6.1.1 Waveform 5.1: Atomic Launch....34
6.2 DESC-LOAD region (region 1, byte offsets)....35
6.3 HARNESS CSR — control (region 2)....36
6.4 Atomic-launch staging (region 2)....38
6.5 HARNESS CSR — status and observability....39
7 Simulation vs Silicon....41
7.1 The two paths....42
7.2 The cocotb testbench....43
7.3 Why no UART-in-sim here....44
8 Troubleshooting....45
8.1 Current state....45
8.2 Known gotchas....46
8.3 Cannot find the board....47
8.4 Board-fit vs sim geometry....48
8.5 Three different “0x1000”s....49
8.6 Vivado-only package collisions....50

List of Figures

Figure 2.1: RAPIDS Characterization Harness Spine.....16
Figure 3.1: Measurement Pipeline.....24

List of Tables

Table 1: Reference documents....9
Table 2: Revision history....11
Table 3: Boards and tool requirements....13
Table 4: Datapath geometry....14
Table 5: Host address regions....16
Table 6: Key RTL blocks....18
Table 7: On-chip harness stimulus and checkers....19
Table 8: What is measured....23
Table 9: Characterization workloads....25
Table 10: flows-rapids-beats make targets....29
Table 11: Host address regions....32
Table 12: DESC-LOAD registers....34
Table 13: Harness CSR — control registers....35
Table 14: Atomic-launch staging registers....37
Table 15: Harness CSR — status and observability....38
Table 16: The two characterization paths....41
Table 17: Known issues and workarounds....45

List of Waveforms

Waveform 5.1: Atomic Launch..... 33

1 Document Information

This is the operator / developer guide for the Nexys A7 **RAPIDS Characterization** project (`projects/fpga-systems/Genesys2/dma-ip/rapids_beats`). It explains what the project characterizes, how the harness works, how to build / simulate / program / run it, and how the harness CSR is configured.

The device under test is the **RAPIDS beats DMA** (`rapids_beats_top`, from `projects/components/dma-ip/rapids/`). This guide documents how the harness drives it and the harness control surface — not the DMA core's internals.

1.1 References

Reference documents

Source	Title
RTL Design Sherpa	`README.md` (project overview + quick start)
RTL Design Sherpa	`docs/rapids_beats_findings.md` (architecture, timing, golden CRC)
RTL Design Sherpa	`flows-rapids-beats/host/README.md` (AXIL word map + campaign semantics)
RTL Design Sherpa	`reports/perf/README.md` (8-channel line-rate performance)
RTL Design Sherpa	`bin/TBClasses/` (shared harness / RegisterMap collateral)
ARM	AMBA AXI4 / AXI4-Stream / APB Protocol Specifications

1.2 Terminology

SRC / SNK

The two wholly-separate RAPIDS engines: `rapids_src_beats` reads memory and emits to the AXIS network; `rapids_snk_beats` receives from the network and writes memory. They share one APB decode (SRC @ 0x0000, SNK @ 0x1000).

Beats

The fixed-quantum transfer variant of RAPIDS. The Genesys 2 build is `DATA_WIDTH = 256` (32 bytes per beat, AXI4 and AXIS alike) since 2026-09-29; peak per-direction bandwidth = 32 B × 100 MHz = 3.2 GB/s. Perf reports v1.0 to v1.5 measured the earlier 512-bit build (64 B/beat, 6.4 GB/s). The bitstream reports its own geometry in the harness `BUILD` register and the host takes its byte and bandwidth math from there.

Region

The top 4 bits of the host word address (`addr[19:16]`) select one of three regions: DUT-REG (0), DESC-LOAD (1), HARNESS CSR (2).

Golden CRC

The deterministic CRC-32 (Ethernet `0x04C11DB7`, LFSR seed `0xDEADBEEF`) that both the on-chip checkers and the host golden model compute; a pass is a CRC match.

1.3 Revision History

Revision history

Version	Date	Notes
0.90	2026-07-18	Initial project guide
0.91	2026-10-15	The on-chip run timer and CRC verdict (`char_run_timer`, shared with STREAM): register map rows at `0x160`-`0x180`, the methodology and Figure 3.1 name it, troubleshooting no longer calls the generator valid flag a non-fatal flake.
0.92	2026-10-20	Block diagrams redrawn to the repo element key (v3): class fills, double-bordered registered blocks, green return paths and a generated key under each figure. No technical content changed.
0.93	2026-10-20	Section (h2) headings now open a new page, the pipeline default. No technical content changed.

2 Overview

2.1 What it characterizes

This project performs on-silicon characterization of the **split RAPIDS beats DMA** (`rapids_beats_top`) — refactored into two wholly-separate engines behind one shared APB decode:

- `rapids_src_beats` — read-only: memory → AXIS network
- `rapids_snk_beats` — write-only: AXIS network → memory

The harness drives both data paths on real hardware over UART and validates every beat against a deterministic golden CRC-32. It reports per-direction throughput and channel scaling, with an all-on-chip stimulus/checker so the board runs at line rate without a host bottleneck.

There is a single flow, `flows-rapids-beats/` (the “beats” fixed-quantum variant of RAPIDS).

2.2 Boards and tools

Boards and tool requirements

Item	Detail
Default board	Digilent Nexys A7-100T (`xc7a100tcsg324-1`), 100 MHz, `NUM_CHANNELS = 4`
Optional board	Genesys 2 (`xc7k325tffg900-2`) via `BOARD=genesys2`, 8 channels (200→100 MHz MMCM)
Build	Vivado (synth / impl / bitgen / program)
Host	Python 3 + `pyserial`
Sim	Verilator + cocotb / cocotb_test / pytest

2.3 Datapath geometry

Datapath geometry

Parameter	Value
Data width	256 bits (32 B/beat), AXI4 and AXIS; 512 bits (64 B) through perf report v1.5
Address width	64 bits
Descriptor	fixed 256-bit
SRAM per channel	128 beats = 4 KB (was 256 x 64 B = 16 KB)
Peak per direction	3.2 GB/s (32 B × 100 MHz)
Peak full-duplex	6.4 GB/s

2.4 Status

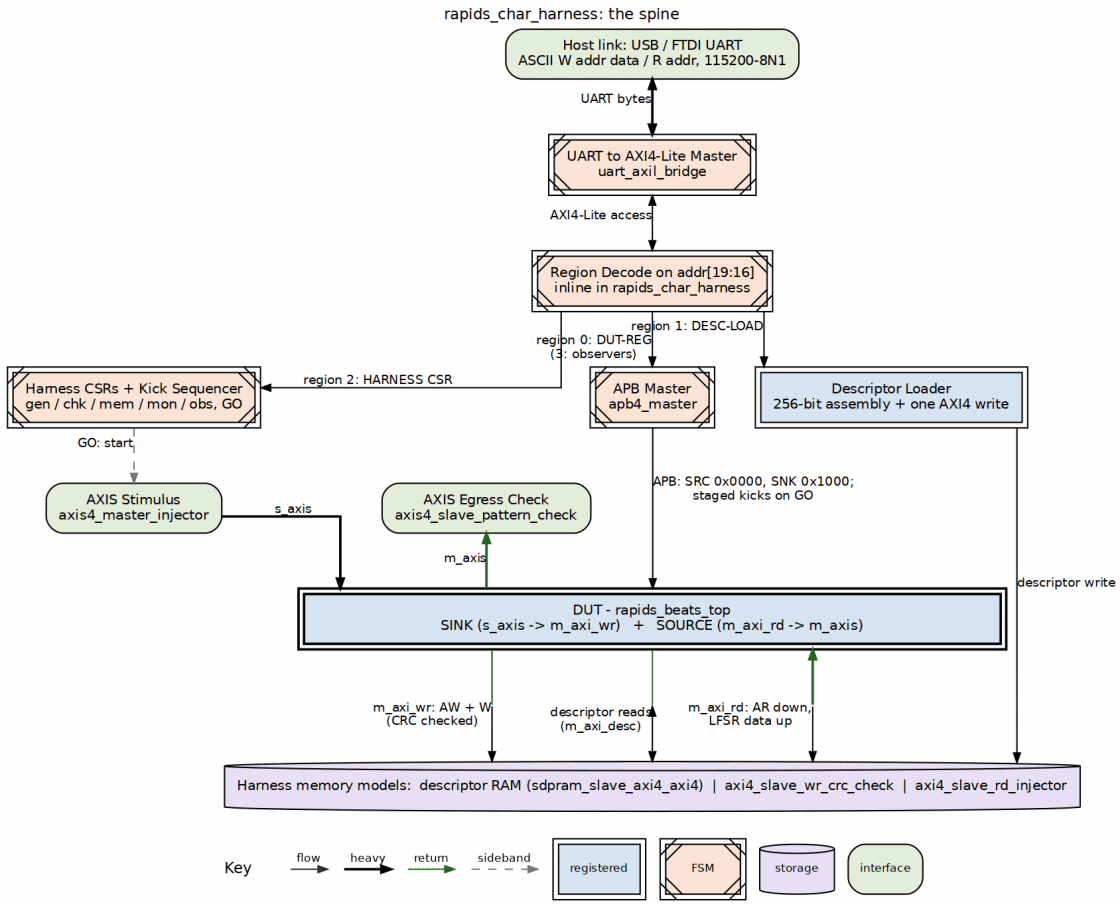
Silicon-validated: `make smoke` passes both paths; `make suite` passes 48/48 (channels {1,2,4} × beats {1,4,8,16} × backpressure {off,on} × 2 seeds). Timing closes at 100 MHz (WNS +0.007 ns, 0 failing endpoints) at the board-fit geometry `NUM_CHANNELS = 4`, `SRAM_DEPTH = 256`. On Genesys 2 the 8-channel 512-bit build reached 99.8–100% line rate (6.40 GB/s per direction, perf report v1.5); the 256-bit / 4 KB-per-channel build is the current design point (Makefile `DATA_WIDTH` / `SRAM_DEPTH`, exported to the build and to `verify-sim`) — see `reports/perf/`.

3 How It Works

3.1 The spine and region decode

Host commands arrive over one UART link and are steered by the top 4 bits of the word address (`addr[19:16]`) into four regions:

3.1.1 Figure 2.1: RAPIDS Characterization Harness Spine



RAPIDS characterization harness spine

Source: 01_harness_spine.dot

Host address regions

Region	Base	Contents
0 — DUT-REG	0x0_0000	AXIL → `apb4_master` → the DUT's APB (SRC @ 0x0000, SNK @ 0x1000)

Region	Base	Contents
1 — DESC-LOAD	0x1_0000	256-bit descriptor assembly + single-beat AXI4 write into descriptor RAM
2 — HARNESS CSR	0x2_0000	gen / chk / mem / mon / obs control + status readback
3 — OBSERVERS	0x3_0000	`USE_OBSERVERS=1` only: the shared interface observers' `obs_regs` maps, AXI observer @ 0x0000, AXIS observer @ 0x1000 (`paddr[12]`); reads 0 with PSLVERR when not built

The UART wire protocol is ASCII (`W <addr> <data>\n` / `R <addr>\n`), 115200 8N1, decoded by `uart_axil_bridge`.

3.2 Key RTL blocks

Key RTL blocks

File	Role
`flows-rapids-beats/rtl/rapids_char_top.sv`	FPGA pin-level top: pins to harness ports, LEDs and 7-seg. It no longer owns the host front-end: that moved into `rapids_char_harness` on 2026-09-23, so the harness simulation drives the board's own launch path.
`flows-rapids-beats/rtl/rapids_char_harness.sv`	Synthesizable harness: the host front-end (`uart_axil_bridge`, the region decode, `apb4_master`, the DESC-LOAD path, the harness CSR register file and the atomic-launch kick sequencer; all CSR/DESC offsets are `localparams` here, the single source of truth for the host regmaps), the DUT `rapids_beats_top`, and the on-chip stimulus / checkers / memories. **This is the cocotb DUT top.**
`flows-rapids-beats/rtl/rapids_char_genesys2_top.sv`	Genesys 2 wrapper (MMCM 200→100 MHz) around `rapids_char_top`, default 8 channels.

3.3 The DUT and the on-chip harness

The DUT is `rapids_beats_top` (`u_dut`), from `projects/components/dma-ip/rapids/`. Two build-time overrides matter for the char build: `USE_AXI_MONITORS = 0` and `GEN_MON = 0` — the in-core AXI/descriptor monitors and MonBus egress are compiled **out** so utilization is metered externally and 8-channel timing closes. Two more knobs, also default off, add the shared interface observers beside the DUT's data ports (rapids TASK-001): `USE_OBSERVERS = 1` builds `axi4_intf_master_observer` on the rd/wr data masters and `axis4_intf_observer` on `s_axis` / `m_axis` (port 0 sink ingress, port 1 source egress), each on its own 4 KB `obs_regs` window in region 3; `OBS_ENABLE_MON_TAPS = 1` arms their monbus event taps (the meters and latency histograms count without them). Their monbus egress lands in an always-accept responder, like the DUT's own; the bare `axi_bus_meter` / `axis_bus_meter` instances and the region-2 `OBS_*` CSRs the host tools read are unchanged. The harness also carries STREAM's memory-latency model: `axi_response_delay` on the source read's R channel and the sink write's B channel, programmed by the `RESP_DELAY` CSR (region 2, 0xC4) in aclk cycles, so the same latency-vs-throughput knee STREAM reports can be swept here (`--suite-delay`).

Measured 2026-09-27 (synth, 8ch, 100 MHz): the pair adds 10,943 LUTs as instruments and 18,733 with the taps armed over the 60,651-LUT bare build, and timing closes in every variant (+3.703 / +0.869 ns synth WNS vs +3.683).

Because all data paths are on-chip, the board runs at line rate with no host bottleneck. The harness instantiates (all sharing LFSR `0xDEADBEEF`, taps `{32,22,2,1}`, CRC-32 `0x04C11DB7`):

On-chip harness stimulus and checkers

Block	Role
`axis4_master_injector`	drives the DUT `s_axis` (sink ingress stimulus)
`axis4_slave_pattern_check`	checks the DUT `m_axis` (source egress)
`axi4_slave_rd_injector`	backs the DUT `m_axi_rd` (`DATA_WIDTH`-bit source data; 256 on the Genesys 2 build)
`axi4_slave_wr_crc_check`	backs the DUT `m_axi_wr` (`DATA_WIDTH`-bit sink data verify)
`sdpram_slave_axi4_axi4` ×4	descriptor + control RAM per half (DUT reads, host writes)
`axi_bus_meter` / `axis_bus_meter`	prod/bp/starv/idle buckets + exact AXIS byte/packet counters

3.4 Atomic launch

To start a measured run coherently, the host stages the kick parameters (`CSR_KICK_MASK`, `CSR_KICK_BASE_*`, `CSR_KICK_STRIDE`) and then writes a single `CSR_GO`, which arms the bus meters, starts the generator, and fires the channel kicks together (Chapter 5). A run latches PASS (`0x0123`) or FAIL (`0x9999`) on the LEDs and 7-seg once `gen_done & src_idle & snk_idle`.

4 Test Methodology

This chapter describes what is exercised on the FPGA block (Figure 2.1) and how throughput and data integrity are measured.

4.1 What is under test

The split RAPIDS beats DMA — the `rapids_src_beats` (read → AXIS) and `rapids_snk_beats` (AXIS → write) engines. Because all stimulus, checking, and memory are on-chip, the board runs at line rate and every measurement is taken at the DUT's own AXI/AXIS ports.

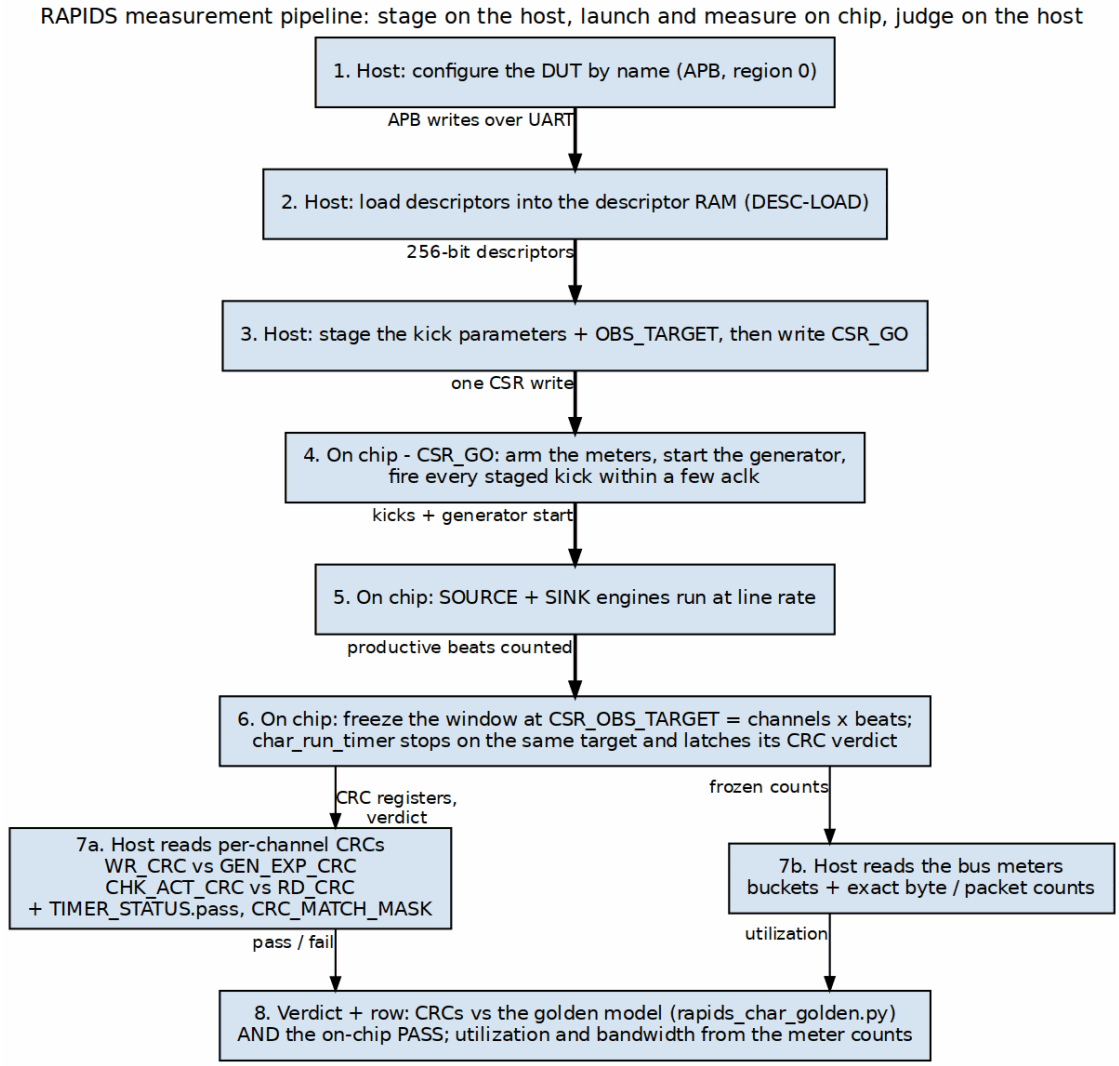
4.2 What is measured, and how

What is measured

Quantity	How it is obtained
Data integrity	Per-channel golden CRC-32: SINK `WR_CRC[ch]` vs `GEN_EXP_CRC[ch]`; SOURCE `CHK_ACT_CRC[ch]` vs `RD_CRC[ch]`; plus `STATUS.data_error`
Throughput	AXIS/AXI bus meters + exact byte/packet counters (`OBS_SIN/SOUT_BYTES/PKTS`) over the timed window
Utilization	`OBS_{RD,WR,SIN,SOUT}_{PROD,BP,STARV,IDLE}` buckets
Scaling	Repeat across the active-channel mask
Observer telemetry (`USE_OBSERVERS=1`)	Region-3 `obs_regs` by name: per-port cycle buckets, AXIS bytes/packets, AXI burst counts and latency histograms, tap drop counts (`OBS_STICKY`)
Sweep axes	`--suite-channels`, `--suite-beats`, `--suite-descs` (chains), `--suite-xfer` (burst AxLEN), `--suite-delay` (`RESP_DELAY`) – STREAM's five knobs

A measurement run uses atomic launch (Waveform 5.1): configure the DUT by name over APB, load descriptors into the per-half descriptor RAM, stage the kick parameters (`CSR_KICK_MASK`, `CSR_KICK_BASE_*`, `CSR_KICK_STRIDE`), then write a single `CSR_GO` that arms the bus meters, starts the generator, and fires the channel kicks together. `CSR_OBS_TARGET = channels × beats` freezes the meter window deterministically; the host then reads the CRCs and counters.

4.2.1 Figure 3.1: Measurement Pipeline



Measurement pipeline

Source: 02_measurement_pipeline.dot

4.3 Workloads

Characterization workloads

Axis	Values
Channels	1, 2, 4 (Nexys) / up to 8 (Genesys 2)
Beats	1, 4, 8, 16
Backpressure	off, on (SOURCE only, via `chk_ready_en`)
Seed	default, alternate (SINK only — SOURCE read-gen is fixed `0xDEADBEEF`)

4.4 The oracle

Every beat is checked against a deterministic **golden CRC-32** (LFSR seed `0xDEADBEEF`, taps `{32,22,2,1}`, poly `0x04C11DB7`). The host's `rapids_char_golden.py` computes the same value the on-chip checkers do; a pass is a match on both paths (`make suite` = 48/48).

The harness also judges each run on chip. Its run timer (`char_run_timer`, the module the STREAM harness uses) compares the active half's input CRC with its output CRC per channel and latches PASS when the run ends. The host requires both: the golden match and the on-chip PASS.

4.5 Measurement pitfalls

- **Arm every run** — the bus meters have no auto-reset; use `OBS_CTRL.arm` or the `CSR_GO` write, or the window stays frozen on the first capture.
- **SINK re-arm** — the sink is fully trustworthy only on the first arm after a board reset; the runner pulses `CHANNEL_RESET` before each run.
- The generator self-CRC (`GEN_EXP_CRC`) is one side of the on-chip SINK verdict. Its valid flag read 0 intermittently in the 2026-07-15 board runs, before the generator start became a one-cycle pulse; no board run since shows it. A recurrence now fails the run instead of logging a warning.

5 Build and Run

All commands assume the Python environment is sourced (the Makefile hard-fails without REPO_ROOT):

```
cd /path/to/RTLDesignSherpa && source env_python
cd projects/fpga-systems/Genesys2/dma-ip/rapids_beats/flows-rapids-beats
```

5.1 Make targets (`flows-rapids-beats/Makefile`)

flows-rapids-beats make targets

Target	What it does
`make sim`	cocotb harness self-check (sink + source, multi-channel)
`make verify-sim`	pre-bitstream sim gate (`-k sink`); bypass with `BITSTREAM_SKIP_VERIFY=1`
`make project` / `make synth`	create Vivado project / synth-only + utilization
`make bitstream`	full synth + impl + bitgen (depends on `verify-sim`)
`make program`	flash the board over JTAG
`make smoke`	fast golden-validated UART confidence check
`make suite`	full UART sweep → JSON under `reports/`
`make characterize`	full UART campaign on `PORT`
`make flow`	sim → bitstream → program → characterize
`make utilization` / `make timing`	print the latest reports
`make clean` / `make clean-all`	remove artifacts

Knobs: `CHANNELS` (4 on nexys, 8 on genesys2 — passed as both the build generic and the host `--channels`, kept in lockstep), `BEATS=8`, `PORT=/dev/ttyUSB1`, `BOARD=nexys|genesys2`, `BITSTREAM_SKIP_VERIFY=1`.

```
make sim                                  # cocotb self-check
make bitstream                            # Nexys A7, 4 channels
make bitstream BOARD=genesys2 CHANNELS=8  # Genesys 2, 8 channels
make program
make smoke PORT=/dev/ttyUSB1
make suite CHANNELS=4
```

The Vivado build targets part `xc7a100tcsg324-1`, top `rapids_char_top`, with board-fit generics `NUM_CHANNELS=4`, `SRAM_DEPTH=256`, `DESC_RAM_ENTRIES=256`. The Genesys 2 build (`BOARD=genesys2`) is 8 channels at the 256-bit / 4 KB per channel design point: Makefile `DATA_WIDTH=256 SRAM_DEPTH=128`, exported as the `RAPIDS_DATA_WIDTH` / `RAPIDS_SRAM_DEPTH` generics and handed to `verify-sim`, so the sim gate builds the same geometry as the bitstream. Override both on the `make` line to build another point; the host reads the result back from the harness `BUILD` register and refuses a `--channels` that disagrees with it. Programming is pinned to a specific JTAG serial so the flash and the UART campaign land on the same board (override `RAPIDS_CHAR_JTAG_SERIAL`).

5.2 The host campaign (`flows-rapids-beats/host/`)

The port defaults to `--port auto`, which probes each `/dev/ttyUSB*` and keeps the board whose region-2 `CTRL/ID` reads `0x52415031` ("RAP1").

`run_characterization.py` — the campaign runner (sink then source):

```
# Full campaign, verbose
./run_characterization.py --port /dev/ttyUSB1 --channels 4 --active 4
--beats 8 -v

# One path only
./run_characterization.py --sink-only   --channels 4
./run_characterization.py --source-only --channels 4

# Sink with the generator round-robining the active channels beat by
beat
# (GEN_MODE.INTERLEAVE): every sink channel holds data at once, so the
# measured write window is the DUT's, not one channel's (rapids TASK-
018)
./run_characterization.py --sink-only --interleave --channels 8 --
active 8

# Smoke (both paths, quick) and full suite (matrix → JSON)
./run_characterization.py --smoke  --channels 4
./run_characterization.py --suite  --suite-channels 1,2,4 --suite-
beats 1,4,8,16 \
        --suite-bp off,on --suite-seeds default,0x12345678
```

Exit codes: `0` all pass, `1` a self-check failed, `2` no UART link / wrong ID.

`dump_status.py` — a STATUS / totals / CRC snapshot (`./dump_status.py --port /dev/ttyUSB1 --channels 4`). Other host modules: `descriptor_builder.py` (256-bit descriptor builder), `rapids_char_golden.py` (standalone golden CRC), `rapids_char_io.py` (transport + autodetect).

5.3 Typical sequence

```
make sim                          # prove the harness self-check
make bitstream                    # (runs verify-sim first)
make program
make smoke PORT=/dev/ttyUSB1      # golden-validated link check
make suite CHANNELS=4             # full sweep -> reports/*.json
```

6 Harness CSR Configuration

6.1 Regions and by-name access

The host address splits into four regions by `addr[19:16]`:

Host address regions

Region	Base	Contents
DUT-REG	0x0_0000	APB into the RAPIDS DUT (SRC 0x0000, SNK 0x1000)
DESC-LOAD	0x1_0000	descriptor holding registers + kick
HARNESS CSR	0x2_0000	gen / chk / mem / mon / obs + status
OBSERVERS	0x3_0000	`USE_OBSERVERS=1` builds only: `obs_regs` of the AXI observer @ 0x0000 and of the AXIS observer @ 0x1000 (`projects/components/utility-ip/misc/rdl/obs/obs_regs.rdl`, by name through `obs_regs_top_regmap.py`); PSLVERR + 0 when not built

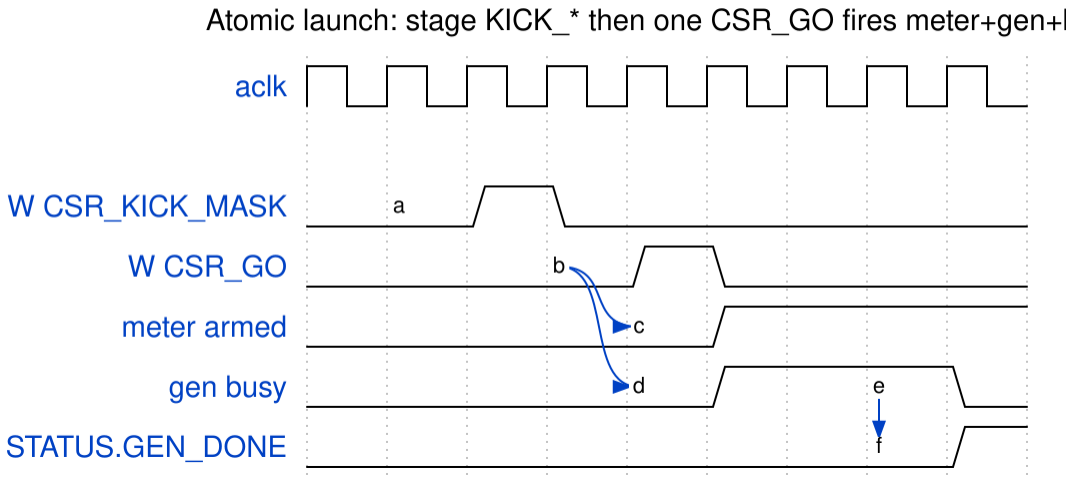
No PeakRDL RDL exists for the harness CSR. The registers are decoded by hand in `rapids_char_top.sv` (localparams) and mirrored by two hand-maintained Python regmaps, regenerated from the SV by `flows-rapids-beats/bin/gen_rapids_harness_regmap.py`:

- `flows-rapids-beats/rtl/rapids_harness_csr_regmap.py`
- `flows-rapids-beats/rtl/rapids_harness_desc_regmap.py`

The **DUT** (RAPIDS core) does use PeakRDL — `projects/components/dma-ip/rapids/rtl/rapids_regmap.py` — loaded by the host as two `RegisterMap` instances (SRC `start_address=0x0000`, SNK `0x1000`, 13-bit APB) for by-name DUT config.

The identity register: region-2 `CTRL/ID` @ 0x000 reads **`0x5241_5031`** (“RAP1”) — the BUILD_ID equivalent used for `ping()` and port autodetect.

6.1.1 Waveform 5.1: Atomic Launch



Atomic launch sequence

Source: 01_atomic_launch.json

6.2 DESC-LOAD region (region 1, byte offsets)

DESC-LOAD registers

Register	Offset	Field	Bits	Access	Description
DESC_WORD0. .7	0x00–0x1C	value	[31:0]	RW	Words of the 256-bit descriptor
DESC_ADDR	0x20	value	[31:0]	RW	Target byte address in descriptor RAM
DESC_KICK	0x24	half	[0]	W	0 = SRC, 1 = SNK — write issues one AXI4 write
DESC_STATUS	0x28	ok	[0]	R	Last BRESP was OKAY

6.3 HARNESS CSR — control (region 2)

Harness CSR — control registers

Register	Offset	Field	Bits	Access	Description
CTRL	0x00	cam_clear	[0]	W	Pulse: clear the sticky-error CAM
GEN_CTRL	0x10	gen_start	[0]	W	Pulse: start the AXIS generator
GEN_SEED	0x14	value	[31:0]	RW	LFSR seed (SINK honors it)
GEN_NBEATS	0x18	value	[31:0]	RW	Beats per channel
GEN_BPP	0x1C	value	[31:0]	RW	Beats per packet
GEN_CHMASK	0x20	value	[N-1:0]	RW	Active-channel mask
GEN_TDEST	0x24	value	—	RW	AXIS TDEST
GEN_MODE	0x28	interleave	[0]	RW	1: round-robin the active channels beat by beat (every sink channel holds data at once); 0: one channel's whole run, then the next
CHK_CTRL	0x30	chk_start	[0]	RW	Pulse: start the checker
CHK_CTRL	0x30	chk_ready_en	[1]	RW	Level: checker backpressure enable
CHK_SEED	0x34	value	[31:0]	RW	Checker seed
MEM_CTRL	0x40	rd_crc_lfsr_reset	[0]	W	Pulse: reset read-CRC LFSR
MEM_CTRL	0x40	wr_crc_reset	[1]	W	Pulse: reset write-CRC

Register	Offset	Field	Bits	Access	Description
MON_BASE / MON_LIMIT	0x50 / 0x54	value	[31:0]	RW	DUT monitor egress window (host default 0x1000 / 0x4FFF)
MON_FLUSHWM	0x58	value	[15:0]	RW	Monitor flush watermark
CH_SEL	0x60	value	—	RW	Selects the channel for indexed CRC reads
OBS_CTRL	0xC0	arm	[0]	W	Pulse: re-arm the bus meters
RESP_DELAY	0xC4	rd_delay / wr_delay	[15:0] / [31:16]	RW	Memory-latency model (STREAM knob 5): per-beat hold, in aclk cycles, injected by axi_response_delay on the source read's R channel and the sink write's B channel. 0 = one register stage.

6.4 Atomic-launch staging (region 2)

These raw offsets are declared in `rapids_char_top.sv` and mirrored in `run_characterization.py` (they are not in the by-name regmap):

Atomic-launch staging registers

Register	Offset	Field	Bits	Description
CSR_KICK_CFG	0x64	half / start_gen_on_go	[0] / [1]	Which half; start gen on GO
CSR_KICK_MASK	0x68	mask	[N-1:0]	Kick channel mask
CSR_KICK_BASE_LO/HI	0x6C / 0x70	value	[31:0]	Descriptor base address
CSR_KICK_STRIDE	0x74	value	[31:0]	Per-channel byte stride
CSR_GO	0x78	go	[0]	Pulse: arm meter + start gen + fire kicks
CSR_OBS_TARGET	0x7C	value	[31:0]	Freeze window at N productive beats (0 = off)

6.5 HARNESS CSR — status and observability

Harness CSR — status and observability

Register	Offset	Field	Bits	Description
STATUS	0x80	mon_irq, src_idle, snk_idle, gen_busy, gen_done, data_error, rd_mem_busy, wr_mem_busy	[0]…[7]	Harness state
GEN_BEATS_T / CHK_BEATS_T	0x84 / 0x88	value	[31:0]	Gen / checker beat totals
PKT_CNT	0x8C	value	[31:0]	Packet count
RD_BEATS_T / WR_BEATS_T	0x90 / 0x94	value	[31:0]	Read / write beat totals
SRC_SCHERR / SNK_SCHERR	0x98 / 0x9C	value	[N-1:0]	Per-channel scheduler error
GEN_EXP_CRC / CHK_ACT_CRC	0xA0 / 0xA4	value	[31:0]	CRC for the CH_SEL channel
RD_CRC / WR_CRC	0xA8 / 0xAC	value	[31:0]	Read / write CRC for CH_SEL
*_VLD	0xB0–0xBC	value	[N-1:0]	Per-channel CRC valid masks
OBS_RD_* / OBS_WR_*	0x100–0x11C	value	[31:0]	AXI4 read / write meter buckets
OBS_SIN_* / OBS_SOUT_*	0x120–0x13C	value	[31:0]	AXIS in / out meter buckets
OBS_SIN_BYTES_LO/HI, OBS_SIN_PKTS	0x140–0x148	value	[31:0]	Exact AXIS-in bytes / packets
OBS_SOUT_BYTES	0x14C–0x154	value	[31:0]	Exact AXIS-out bytes /

Register	Offset	Field	Bits	Description
_LO/HI, OBS_SOUT_PKTS				packets
TIMER_STATUS	0x160	done, running, pass, crc_any_valid, crc_match, verdict_valid	[0]…[5]	On-chip run timer and CRC verdict (char_run_timer); pass is latched at done
TIMER_CYC_LO/HI	0x164 / 0x168	value	[31:0]	Run length in aclk cycles, GO to the OBS_TARGET-th output beat
TIMER_R_FIRST/R_LAST/W_FIRST/W_LAST	0x16C–0x178	value	[31:0]	Cycle stamps of the first and last input-side and output-side beat (low word)
CRC_VALID_MASK / CRC_MATCH_MASK	0x17C / 0x180	value	[N-1:0]	Per channel: both CRCs of the pair valid / valid and equal

Meter buckets and byte/packet counters are cleared by CTRL.cam_clear and each run must be armed (via OBS_CTRL.arm or the CSR_GO write) or the window stays frozen on the first capture.

The run timer is the same char_run_timer the STREAM harness uses (projects/components/utility-ip/misc/rtl/char_run_timer.sv; BUILD.RUN_TIMER says it is built). GO clears it and starts it a cycle later; it stops when the output-side beat count reaches OBS_TARGET. Its verdict compares the active half's input CRC with its output CRC: on SOURCE, RD_CRC against CHK_ACT_CRC; on SINK, GEN_EXP_CRC against WR_CRC. The host fails a run when the timer did not finish, when pass is clear, or when an active channel is missing from CRC_MATCH_MASK.

7 Simulation vs Silicon

RAPIDS characterization reaches the **same golden CRC** two ways — but, unlike the ddr2/stream siblings, **the simulation does not run the host over a simulated UART.** The UART bridge and CSR router live only in the board top (`rapids_char_top.sv`); the cocotb DUT top is the harness (`rapids_char_harness`), and the testbench drives its control surface directly.

7.1 The two paths

The two characterization paths

	Silicon	Simulation
Config	host `RegisterMap` over UART → APB	cocotb `RegisterMap` over the harness APB port
Descriptors	host DESC-LOAD region → descriptor RAM	cocotb 256-bit AXI4 host-write port
Kick	host `CSR_GO` atomic launch	cocotb per-half kick windows
Data paths	all on-chip (harness gen/check/mem)	the same on-chip blocks, elaborated in Verilator
Pass criterion	golden CRC match (host `rapids_char_golden.py`)	golden CRC match (checker CRC == expected)

The equivalence is by **shared logic**, not a shared transport: the same by-name DUT config (`rapids_regmap.py`), the same descriptor bytes (the TB's `create_descriptor()` mirrors `flows-rapids-beats/host/descriptor_builder.py`), and the same golden CRC-32 asserted in both places.

7.2 The cocotb testbench

- **DUT top:** `rapids_char_harness` (Verilator). Runner `flows-rapids-beats/dv/test_rapids_char_harness.py`; TB class `RapidsCharHarnessTB` (`flows-rapids-beats/dv/rapids_char_harness_tb.py`).
- **Two self-checks:**
  - `cocotb_test_sink_selfcheck` — `s_axis → sink → m_axi_wr`; asserts `wr_crc[ch] == gen_expected_crc[ch]` for each active channel.
  - `cocotb_test_source_selfcheck` — `m_axi_rd → source → m_axis`; asserts `chk_actual_crc[ch] == rd_crc[ch]` and `data_error == 0`.
- **Params:** `NUM_CHANNELS=8` (env `TEST_NUM_CHANNELS`), `NUM_ACTIVE=4`, `NUM_BEATS=8`, `DATA_WIDTH=512`, `SRAM_DEPTH=512` when run bare; `make sim` / `make verify-sim` pass the build's `DATA_WIDTH` / `SRAM_DEPTH` (256 / 128 on the Genesys 2) as `TEST_DATA_WIDTH` / `TEST_SRAM_DEPTH`. Waves via `WAVES=1`.
- **Gate:** `make verify-sim` runs the sink self-check before a bitstream build, so a broken elaboration is caught before a long Vivado run.

7.3 Why no UART-in-sim here

The harness data paths are entirely on-chip, so there is nothing for a simulated host to *drive* beyond the control surface — the TB pokes that surface directly (APB + descriptor RAM + kick) and reads back the same CRCs the board's host reads over UART. If a UART-level equivalence sim is wanted later, the shared `make_uart_channel` / `CocotbUartChannel` collateral in `bin/TBClasses/harness/` (used by ddr2/stream) can wrap the board top — but it is not wired today.

8 Troubleshooting

8.1 Current state

Silicon-validated. `make smoke` passes both paths; `make suite` passes 48/48 (3 channels × 4 beats × 2 backpressure × 2 seeds). Timing closes at 100 MHz (setup WNS +0.007 ns, 0 failing endpoints; hold WHS +0.011 ns). Post-route utilization (4-channel, `xc7a100t-1`): 37,555 LUTs (59.2%), 28,683 FF (22.6%), 22 BRAM (16.3%), 0 DSP. There is no separate `KNOWN_ISSUES` file — the known-state material lives in `docs/rapids_beats_findings.md`.

8.2 Known gotchas

Known issues and workarounds

Symptom	Cause / workaround
SINK over-produces / desyncs	The generator `start` must be a one-cycle pulse, not a held level (a UART-held level runs the generator repeatedly). Fixed in RTL; residual: SINK is fully trustworthy only on the **first arm after a board reset** — a full SINK sweep needs a per-config reset. SOURCE re-arms cleanly.
Stale scheduler/descriptor state	`reset_channels()` pulses `CHANNEL_RESET` on both halves before each run — the sink does not self-clean after a transfer.
Meter reads 0 / frozen	The bus meters have no per-run auto-reset. Arm each run via `OBS_CTRL.arm` or the `CSR_GO` write; use `CSR_OBS_TARGET = channels × beats` for a deterministic freeze window.
`gen_expected_crc_valid` intermittently 0	Seen only in the 2026-07-15 board runs, before the generator start became a one-cycle pulse; none since. The on-chip SINK verdict now needs it, so a recurrence fails the run (`hardware CRC verdict FAIL`) instead of warning. Treat it as a bug to find, not a flake.
`hardware timer not done`	GO did not start the run timer, or the output-side beat count never reached `OBS_TARGET`. Check `OBS_TARGET` = channels x beats and the beat totals.
Spurious read failure	`csr_read()` retries up to 3× on malformed/short UART frames.

8.3 Cannot find the board

The USB-UART re-enumerates across reboots/replugs. `--port auto` (default) probes each `/dev/ttyUSB*` and picks the board whose region-2 `CTRL/ID` reads `0x52415031` (“RAP1”). `make program` and the UART campaign are pinned to the same board by JTAG serial (override `RAPIDS_CHAR_JTAG_SERIAL`) so the flash and the run land together.

8.4 Board-fit vs sim geometry

The harness sim defaults (`SRAM_DEPTH = 4096`, `DESC_RAM_ENTRIES = 2048`) inflate to ~148 BRAM tiles — over the 100T's 135 — so the board build trims both to 256 (~22 tiles). This is a build-time generic only; the RTL is unchanged. Set `CHANNELS` once (it drives both the build generic and the host `--channels`) so the two never drift.

8.5 Three different “0x1000”s

Do not conflate them: the **SNK APB half** is at APB `0x1000`; the host default **`MON_BASE`** is `0x1000`; and the in-core DUT monitors (which would live in a `0x1000` monitor window) are compiled **out** (`GEN_MON = 0`) for this build.

8.6 Vivado-only package collisions

`rapids_pkg` vs `stream_pkg` enum-label/type clashes (`RD_*`, `CH_*`, `channel_state_t`, `descriptor_t`, ...) surface only under Vivado's flattened `$unit` scope — Verilator does not catch them. They are resolved by `rapids_pkg::` qualification; if a Vivado elaboration errors on a duplicate enum label, look here first.